VBA Notes

Dictionary Object — Complete Reference

What is a Dictionary in VBA?

A Dictionary is an object from the Scripting Runtime library that stores key–value pairs.

Think of it as a phone book:

- **Key** = Person's name
- **Value** = Phone number

Key rules to remember:

- Each key must be **unique**, but values can repeat.
- You must create it using `CreateObject("Scripting.Dictionary")` or add a reference to **Microsoft Scripting Runtime**.

Example 1 — Simple Dictionary

Create a basic dictionary, add key-value pairs, retrieve a value, and loop through all entries.

```
Sub SimpleDictionary()
    Dim dict As Object
    Set dict = CreateObject("Scripting.Dictionary")

    ' Add key-value pairs
    dict.Add "A1", "Akshay"
    dict.Add "A2", "Snehal"
    dict.Add "A3", "Rohit"

    ' Retrieve value using key
    MsgBox "Key A2 = " & dict("A2")

    ' Loop through dictionary
    Dim key As Variant
    For Each key In dict.Keys
        Debug.Print key & " -> " & dict(key)
    Next key
End Sub
```

Example 2 — Updating and Removing Items

Shows how to update an existing value and remove a key from the dictionary.

```
Sub UpdateRemoveDictionary()
```

```
    Dim dict As Object
    Set dict = CreateObject("Scripting.Dictionary")

    dict.Add "E1", "Excel"
    dict.Add "P1", "Power BI"

    ' Update value
    dict("E1") = "Excel VBA"

    ' Remove one item
    dict.Remove "P1"

    ' Count items
    MsgBox "Dictionary Count = " & dict.Count
    Debug.Print dict.Count
End Sub
```

Example 3 — Checking Keys (Exists Method)

Use the Exists method to safely check if a key is present before accessing it.

```
Sub CheckKeys()
    Dim dict As Object
    Set dict = CreateObject("Scripting.Dictionary")

    dict.Add "U1", "User1"

    If dict.Exists("U1") Then
        MsgBox "Key U1 exists -> Value = " & dict("U1")
    Else
        MsgBox "Key U1 not found"
    End If
End Sub
```

Example 4 — Counting Frequencies

A very common use of Dictionary is counting how many times each value appears in a list.

```
Sub CountFruits()
    Dim dict As Object
    Set dict = CreateObject("Scripting.Dictionary")

    Dim fruits As Variant
    fruits = Array("Apple", "Banana", "Apple", "Orange", "Banana", "Apple")

    Dim f As Variant
    For Each f In fruits
        If dict.Exists(f) Then
            dict(f) = dict(f) + 1
        Else
            dict.Add f, 1
```

```
        End If
    Next f

    ' Show results
    Dim k As Variant
    For Each k In dict.Keys
        Debug.Print k & " -> " & dict(k)
    Next k
End Sub
```

Key Features — Dictionary vs Collection

Comparison of the two most common VBA data structures:

Feature	Collection	Dictionary
Keys must be unique	✓Yes	✓Yes
Retrieve by key	✓Yes	✓Yes
Retrieve by index (position)	✓Yes	✗No
Check if key exists	✗No	✓Yes (.Exists)
Update values directly	✗Harder	✓Easy (dict(key) = value)
Faster lookups	✗Slower	✓Faster

Summary

Dictionary = advanced key-value store in VBA.

- **Best for:** counting, mapping, fast lookups, avoiding duplicates.
- Use `.Exists` to check before adding — prevents runtime errors.
- Use `.Keys` and `.Items` for looping through all entries.

Quick Reference — Dictionary Methods & Properties

Method / Property	Description
`.Add(key, value)`	Adds a new key-value pair
`.Remove(key)`	Removes the specified key
`.Exists(key)`	Returns True if key exists

Method / Property	Description
`.Count`	Returns total number of items
`.Keys`	Returns array of all keys
`.Items`	Returns array of all values
`dict(key) = value`	Updates value for an existing key
`.RemoveAll`	Clears all items from the dictionary